

AnalyzerService / Sysmex CS-2000i

Руководство по работе службы, размещению общих библиотек, обработке результатов и сборке. Версия 06.10.2026; .NET 9 / Windows Service.

1. Назначение и границы ответственности

AnalyzerService.Host читает JSON, загружает плагины и управляет их жизненным циклом. В **AnalyzerRuntime** нет знаний о TCP, COM, файловом обмене и формате сообщений. Конкретный **SysmexCS2000.HostOnline.Driver** открывает TCP listener, принимает кадры Host Online, запрашивает задания в ЛИС и сохраняет результаты. Общий **AnalyzerService.Transport** предоставляет TCP-host как библиотеку, но решение использовать его принимает только драйвер.

Общие сборки Contracts, Transport, LisDatabase, ResultFiles и их пакетные зависимости включены в однофайловую публикацию Host. Это **зависимости упаковки**, а не вызовы транспорта из AnalyzerRuntime. Внешняя DLL Sysmex здесь не используется: DLL драйвера - собственный управляемый плагин протокола Host Online.

Компонент	Ответственность
Host / Worker / Manager / Runtime	Конфигурация, загрузка, запуск, наблюдение задачи, Stop и Dispose.
DriverLoadContext	Изолирует код плагина, но разделяет четыре общие сборки с Host.
Sysmex-драйвер	TCP listener, R221/S221, D121/D221, журнал, вызовы БД и очереди.
RawResultQueue	Исходные .raw, отдельный поток, archive/errors, сырые QC-файлы.
HostOnlineResultHandler	Сопоставление кодов с ЛИС, числовой формат, .res/.ok.

2. Последовательность запуска

1. Program создаёт Generic Host, регистрирует службы и Worker; Windows Service запускает ExecuteAsync.
2. Worker читает только JSON из верхнего уровня configs, валидирует общие поля и добавляет активные анализаторы в AnalyzerManager.
3. Manager вызывает AnalyzerRuntime.StartAsync для каждого прибора. Runtime по DllPath создаёт DriverLoadContext, загружает DLL и экземпляр IAnalyzerDriver.
4. Runtime вызывает Initialize(logger, settings). Sysmex-драйвер проверяет собственные TCP/SQL/файловые поля и создаёт AnalyzerSysmexCS2000HostOnline, LisDBProvider и RawResultQueue.
5. Runtime вызывает RunAsync(token). Драйвер запускает поток очереди результатов, открывает TcpHost по IPAddress/Port из JSON и возвращает Task рабочего цикла. Manager наблюдает задачи всех приборов.

```

Program -> Worker.ExecuteAsync -> JsonAnalyzerSettingsProvider.LoadAll
-> AnalyzerManager.Add / StartAllAsync
-> AnalyzerRuntime.StartAsync -> DriverLoader.Load
-> IAnalyzerDriver.Initialize -> IAnalyzerDriver.RunAsync
-> AnalyzerSysmexCS2000HostOnline.RunConnectionsAsync

```

AcceptAsync и чтение TCP-потока асинхронны, чтобы не занимать поток ожиданием сети. Синхронные SQL-запросы, короткая запись сырого кадра и операции выделенного потока RawResultQueue остаются синхронными. Так устроен текущий код, и это важно учитывать при оценке задержек.

3. Запрос задания R221 и ответ S221

Ниже фрагмент **реального стендового журнала** от 02.10.2026. Пробелы в 15-символьных полях значимы; при копировании текста из PDF их выравнивание может измениться. Для точного кадра используйте Protocol log.

```
RX: R2210101 021026110300000304 9000003160B 400 650 660 870 880
```

1. ReadTextAsync извлекает тело между STX и ETX, записывает RX в Protocol и собирает блоки сообщения.

2. HostOnlineCodec.ParseInquiry проверяет R221, читает штатив, позицию, Sample ID и группы параметров. Driver пишет отдельную диагностическую запись с этими полями.
 3. LisDBProvider.GetOrder находит пациента и незавершённые тесты по Sample ID. Каждому кандидату записывает причину включения или исключения; коды групп не дублируются.
 4. HostOnlineCodec.BuildOrder формирует один или несколько S221 (до 22 кодов на блок), переносит предусмотренное поле имени пациента и параметры. WriteTextAsync добавляет STX/ETX и записывает TX в Protocol.
- Если SQL не нашёл пациента/задание, код ответа формируется существующей логикой драйвера. Дата рождения и пол не имеют отдельного поля в используемом формате S221. ACK/NAK поверх TCP в этой версии не добавлены: стендовая проверка этого поведения была отложена.

4. Получение D121/D221 и поток файлов

Пример пациентского D121 из того же стендового журнала (здесь значения `\*\*\*\*\*` - именно текст, а не число):

```
RX: D1210101U021026114700000504 9000003160BTEST МИХАЛ IVA391*****392*****401*****402*****661*****662*****
```

1. Driver записывает RX, тип D121, штатив, позицию, образец и перечень разобранных результатов. Признак U направляет пациентское сообщение в RawResultQueue.SaveResult.
2. SaveResult атомарно сохраняет **точные байты STX...ETX** в отдельный .raw файл по образцу и будит выделенный поток. Приём следующих сообщений не ждёт преобразования в ЛИС.
3. Выделенный поток подбирает .raw, вызывает ProcessStoredResult -> HostOnlineCodec.ParseResult -> HostOnlineResultHandler.Handle. Кодек применяет масштаб из таблицы Host Online; итог ЛИС округляется до одного десятичного знака и содержит запятую.
4. Обработчик ищет сопоставление с PSMV2 через LisDBProvider, создаёт .res и затем .ok. Успешный .raw переносится в Results/archive; при ошибке, отсутствии результатов или сопоставленных тестов - в Results/errors. Ошибка журналируется вместе с исключением.

Если в поле типа стоит C, сообщение является контролем качества. SaveQualityControl записывает его байты без преобразования в Results/QualityControl; отдельный обработчик QC пока не реализован. Например, на стенде принят D121 с признаком C и тестами 402, 652, 882.

5. Повторные low-результаты 872 и 662

Настройка прибора позволяет после med-измерения отправить low-измерение того же теста. При создании .res код low заменяется **только для поиска сопоставления в ЛИС**. Исходный D121/D221, .raw, код и величина результата прибора остаются прежними.

Код в D-сообщении	Код для поиска PSMV2	Результат
882 (med)	882	Обычная строка .res
872 (low)	882	Дополнительная строка .res того же теста
652 (med)	652	Обычная строка .res
662 (low)	652	Дополнительная строка .res того же теста

Пример, **условные данные**: блок `872 123` кодек интерпретирует как 12.3 по масштабу кода 872; в .res пишется значение `12,3` с кодом PSMV2, найденным по 882. Блок `662 123` обрабатывается аналогично через 652. Если med и low находятся в одном сообщении, обе строки сохраняются в исходном порядке, без удаления дубликата. ЛИС должна допускать два результата с одним кодом теста в файле; это следует проверить при следующем стендовом прогоне.

6. Остановка и исключения

1. Service Control Manager вызывает Worker.StopAsync -> AnalyzerManager.StopAllAsync. Менеджер останавливает каждый Runtime последовательно и собирает ошибки.
2. Runtime отменяет token, вызывает синхронный IAnalyzerDriver.Stop и ожидает Completion. Sysmex Stop закрывает TCP listener и активное соединение, разблокируя AcceptAsync/ReadAsync.

3. RunConnectionsAsync в finally останавливает RawResultQueue после текущего файла; необработанные .raw сохраняются и будут подобраны при следующем запуске.

4. Manager вызывает Runtime.Dispose -> LoadedDriver.Dispose -> DLL.Dispose и DriverLoadContext.Unload. Затем освобождаются токены. Ошибки Stop/Dispose записываются в файловый Error-журнал и агрегируются.

Исключение в обработчике сеанса сначала журналируется драйвером, после чего `throw;` передаёт тот же объект и стек выше: TCP-цикл фиксирует сбой клиента. Ошибки RunAsync, загрузки и остановки доходят до Worker/Manager и также записываются. Для ошибок файловой очереди файл переносится в errors; исключение не теряется из журнала.

7. Изоляция зависимостей и состав поставки

```
publish/AnalyzerService/AnalyzerService.Host.exe
publish/AnalyzerService/configs/SysmexCS2000.json
publish/AnalyzerService/drivers/SysmexCS2000HostOnline/
SysmexCS2000.HostOnline.Driver.dll
SysmexCS2000.HostOnline.Driver.deps.json
```

DriverLoadContext загружает сам Sysmex-плагин из его подпапки. Для Contracts, Transport, LisDatabase и ResultFiles он явно возвращает сборки основного контекста, чтобы не создавать несовместимые копии типов. Остальные специфичные зависимости нового драйвера требуют явного решения по публикации. Host не сканирует папку drivers автоматически: путь главной DLL берётся только изDllPath существующего JSON.

Сборка Host однофайловая, но файл DLL драйвера должен оставаться внешним для динамической загрузки. Публикация framework-dependent: на сервере необходим .NET 9 Runtime. В корне пакета могут быть PDB/XML диагностические файлы; общих DLL рядом с драйвером нет. Native-библиотеки при необходимости извлекаются средой .NET из однофайлового EXE.

8. Сборка, развёртывание и проверка

1. В Visual Studio 2022 установите .NET 9 SDK и откройте AnalyzerService.sln. Выполните Restore NuGet и Build Solution в конфигурации Release / x64.

2. Из PowerShell в корне SysmexCS2000Ready запустите `./package.ps1`. Скрипт публикует Host для win-x64 и копирует в drivers ровно два файла Sysmex-плагина. Запуск повторно очищает только каталог publish проекта.

3. Перенесите весь каталог publish/AnalyzerService на сервер. Настройте configs/SysmexCS2000.json: локальный IPAddress и Port listener, подключение к SQL, OutputFolder, ResultsFolder и DllPath. Структура JSON не менялась.

4. Запускайте EXE как консольное приложение для проверки, затем регистрируйте Windows Service обычным способом. Учётной записи нужны права чтения configs/drivers, записи в результаты, выход ЛИС и журналы, а также доступ к SQL Server.

```
dotnet build AnalyzerService.sln -c Release --maxcpucount:1
dotnet run --project tests/SysmexCS2000.HostOnline.Driver.Tests -c Release --no-build
./package.ps1
./tests/PublishedPackageSmoke.ps1
```

Smoke-тест запускает копию опубликованного Host на loopback, проверяет приём QC и точность сохранённых байтов. Он также отклоняет лишние файлы в папке драйвера. Проверка в реальном контуре всё ещё нужна для SQL-сопоставления 872/662 и поведения ЛИС при двух строках одного теста.